

Forge

Continuation-local synthesis, inductive algebra, and cyclic proof compilation

Concrete algorithms, exact certificates,
executable prototypes, and a Lean integration plan

Prepared for Vladimir Reshetnikov

15 September 2026

Abstract

Forge already has a substantial design for orchestration, proof acceptance, arithmetic witnesses, induction planning, and certificate-producing backends. The useful next step is not another orchestration layer. It is a set of workers that construct proof structure which the existing workers do not yet construct.

This article develops three such extensions. Continuation-local synthesis derives fold algebras from residual specifications and searches their holes separately, with an explicit treatment of correlations. Inductive algebra combines a fast pullback-stable vector-space algorithm with target-seeded ideal completion, which can discover mutually supporting polynomial equations and generate exact orbit counterexamples. Cyclic proof compilation treats a recursive obligation graph as a candidate for ordinary well-founded induction, using synthesized phase-sensitive or lexicographic rankings rather than accepting a circular proof.

The archive contains runnable Python implementations, an independent exact replayer, 135 replayed encoded certificates, 31 rejected corruptions, and detailed experimental records. The results establish the specified finite-fragment algorithms, not an end-to-end Lean tactic. Lean bridge code and proof specimens are explicitly separated from executed work.

Repository-relative additions; not a replacement for Forge's existing article.

Primary sources and repository revisions are pinned in the bibliography and source audit.

Contents

1	The missing capabilities, not another redesign	3
1.1	The reviewed boundary	3
1.2	What the implementation actually establishes	3
1.3	A single operational principle	4
2	Continuation-local synthesis	4
2.1	A concrete composition failure	4
2.2	Recover the carrier from the elimination spine	4
2.3	Derive contracts from a specification, not from a type	5
2.4	The Church representation boundary	6
2.5	The executable search	6
2.6	Do not replace relational compatibility by projections	7
2.7	Source identity, instantiation, and scheduling	7
2.8	What the experiment isolates	7
3	Coupled invariants by descending linear spaces	8
3.1	Conservation is stronger than induction needs	8
3.2	Problem statement	8
3.3	An executable preimage computation	9
3.4	A coupled nonlinear example	9
3.5	A stronger completeness statement for affine transitions	10
3.6	The certificate and its proof cost	10
3.7	Guards and control locations	11
4	Target-seeded inductive ideal completion	11
4.1	The missing case after linear-space closure	11
4.2	A completion algorithm	11
4.3	Why completion avoids the bilinear synthesis trap	13
4.4	A genuinely polynomial action	13
4.5	Traced membership, not a trusted CAS answer	13
4.6	Counterexamples and conservative refusal	14
4.7	Where this lane belongs in the scheduler	14
5	Cyclic obligations compiled to well-founded proofs	14
5.1	A repeated goal is a candidate recursive call	14
5.2	The typed object to compile	15
5.3	Why node phases matter	15
5.4	Permuted arguments and lexicographic growth	16
5.5	A tempting but invalid local rule	16
5.6	Size-change closure as a proposal filter	16
5.7	Exact affine and lexicographic certificates	17
5.8	A concrete production synthesis algorithm	17
5.9	Compiling an SCC without introducing circular declarations	17
6	Putting the workers into Forge	18
6.1	Three new payloads, not a new authority model	18
6.2	A target-directed worker protocol	18
6.3	A cooperative proof trace	19
6.4	Dependent contexts require explicit call morphisms	19
6.5	Reification obligations specific to these algorithms	20
6.6	Concrete module boundaries	20
6.7	Library choices and substitutions	21
6.8	Resource allocation that follows the mathematics	21
7	Development sequence and release gates	21
7.1	First release: exact inductive algebra	21
7.2	Second release: one source-preserving algebra region	22

7.3	Third release: explicit cyclic skeletons	22
7.4	What would constitute a meaningful improvement	22
8	Executed experiments and their evidential limits	22
8.1	Reproducibility envelope	22
8.2	Continuation composition	23
8.3	Inductive algebra	23
8.4	Descent on supplied cyclic graphs	23
8.5	Independent replay and deliberate corruptions	24
8.6	What remains unmeasured	24
9	Conclusion	25
A	Certificate conventions	25
A.1	Exact polynomials	25
A.2	Action records	25
A.3	Ranking records	26
A.4	Orbit records	26
B	Why the finite synthesis counts have the reported shape	26
C	Reproducing the package	26
C.1	A first Lean verification session	27

1 The missing capabilities, not another redesign

1.1 The reviewed boundary

This study uses Forge revision 674521027d96, Leant revision 6bf05ad78c46, and standalone Djex revision e8778f4ebd63. Leant’s own Djex dependency is a different revision, e237e866; it must not be silently replaced by the standalone head when reproducing Leant’s reports. The audit concentrates on Forge’s merged structural-search and integration sections, its provenance appendix, its explicit Leant/Djex review, and the neighboring projects’ current synthesis diagnostics and rank-N description [1, 2, 6, 3, 4, 7, 9].

The repository’s design is already considerably broader than “run a stronger arithmetic tactic.” In particular, induction generalization, demand-driven lemma discovery, arithmetic witness synthesis, negative-evidence discipline, and the separation of proposals from accepted proofs are already there. Recommending them again would not answer the present request. The three extensions below start precisely where those descriptions stop.

Area	Existing boundary	Addition developed here
Higher-order synthesis	Type-directed candidates and higher-order carriers are available; whole compositions can still miss the budget.	Residual-specification-derived algebra contracts, local hole search, and a compatibility join before assembly.
Inductive equations	Polynomial recurrence synthesis and conserved quantities; general coupled ideal preservation is recognized as bilinear.	A descending linear-space algorithm that avoids joint bilinear synthesis, followed by target-directed ideal completion with polynomial multipliers.
Recursive proof search	Induction planning and acyclic tabling; a circular dependency is not a proof.	Typed cyclic obligation candidates compiled through explicit, checked well-founded descent on tagged states.

Table 1: The contribution boundary. Existing material is used as infrastructure, not claimed as a new capability.

The first column of additions is a search decomposition, the second is an algebraic closure computation, and the third is a proof-construction strategy. They are not proposed as new mathematical discoveries. Type-and-example-directed synthesis, refinement-based synthesis, polynomial invariant analysis, and cyclic proof systems are established research directions [10, 11, 12, 13, 16]. The contribution is a concrete selection and adaptation for Forge, including executable algorithms, certificate formats, failure cases, and integration gates.

1.2 What the implementation actually establishes

Evidence status. Executed: finite-fragment Python synthesis; exact rational invariant-space and ideal algorithms; ranking synthesis for supplied call graphs; an independent certificate replayer; differential, mutation, and regression tests. Not executed: a Lean tactic, source-to-polynomial reification, Lean kernel replay, or a production change to Djex or Leant.

The distinction is important because the strongest result here is not a claim that a new `forge` already beats `grind`. There is no such end-to-end measurement. The strongest demonstrated result is that small, precisely delimited workers can produce structures unavailable to simpler ablations: a composition assembled from locally constrained holes; a nonconserved but inductively closed family of polynomials; and a decreasing interpretation of a cyclic call graph.

The source tree includes a proposed Lean file, `lean/Specimens.lean`, with ordinary induction and well-founded-induction lemmas. It contains no `sorry`, but it was not compiled: no Lean executable was

available in this execution environment. Its presence is a starting point for porting, not a checking receipt. This does not change the separate elaboration evidence already recorded by Forge for its own core design files.

1.3 A single operational principle

All three extensions address the same practical failure: local search has found useful pieces but has not discovered how they fit together. The remedy is to make the required interface between pieces explicit.

For a fold, the interface is a base/constructor contract. For a family of invariants, it is an action matrix or a matrix of polynomial multipliers. For recursive obligations, it is a local proof constructor plus a ranking relation. A successful search exports that interface, not just a Boolean success flag.

This yields a useful division of labor. Forge’s existing obligation controller owns the original theorem and invokes these workers. A worker may introduce auxiliary obligations, but those obligations must have a mathematical reason: a fold equation, an inductive closure condition, or a strict-descent requirement. The controller does not need to guess arbitrary extra lemmas to make the worker look productive.

The resulting development order should be: first exact proof lowering for the algebraic certificates; then continuation-local synthesis with a checked fold lemma; then the restricted cyclic compiler. The cyclic compiler depends on more source-level structure than either algebraic lane, and should not be the first component whose correctness is tested through a production tactic.

2 Continuation-local synthesis

2.1 A concrete composition failure

Leant’s current indexing diagnostics are unusually useful because they separate missing expressiveness from poor composition order. The original bounded Haskell query produces 256 checked candidates and falsifies all of them. Separate probes find both function-valued Church folds and the required integer-case step; the step appears at candidate 5 in an isolated context and candidate 219 in a larger context. The report explicitly warns that neither number is the step’s ordinal inside the original whole-term search [8].

The appropriate response is therefore not “add function types to the carrier grammar.” They are already supported. It is to expose a fold’s algebra as a separate synthesis problem, constrain that algebra using the residual specification, and avoid repeatedly discovering its local pieces inside many unrelated whole terms.

2.2 Recover the carrier from the elimination spine

Suppose a candidate expression is obtained by eliminating a polymorphic value and then applying the result to arguments of types $B_1, \dots, B_k$, with final result type C . One useful carrier candidate is

$$R = B_1 \rightarrow \dots \rightarrow B_k \rightarrow C. \quad (1)$$

This is a constraint implied by that particular application spine, not a general solution to higher-rank type inference. Other spines, other carriers, and alternative polymorphic instantiations remain separate search alternatives.

For integer indexing with a supplied default $d : A$, the last application is to an index $i : \mathbb{Z}$, and the observed result has type A . The carrier suggested by (1) is $R = \mathbb{Z} \rightarrow A$. A right fold therefore needs

$$b : \mathbb{Z} \rightarrow A, \quad a : A \rightarrow (\mathbb{Z} \rightarrow A) \rightarrow (\mathbb{Z} \rightarrow A).$$

The nonstructural input has moved *inside* the fold carrier. This is the functional counterpart of generalizing an accumulator before induction, but the search object is now a local algebra rather than a strengthened theorem.

2.3 Derive contracts from a specification, not from a type

Let $J_d(xs, i)$ denote the total indexing operation that returns d at negative or out-of-range indices. Its structural equations are

$$J_d([], i) = d, \quad (2)$$

$$J_d(h :: xs, i) = \begin{cases} d, & i < 0, \\ h, & i = 0, \\ J_d(xs, i - 1), & i > 0. \end{cases} \quad (3)$$

The type alone does not select this behavior: always returning d has the same result type. The equations must come from the user's specification, a proved library characterization, or a separately proved unfolding of the intended operation.

For the selected carrier, the equations induce local algebra contracts:

$$b(i) = d, \quad (4)$$

$$a(h, k)(i) = \begin{cases} d, & i < 0, \\ h, & i = 0, \\ k(i - 1), & i > 0, \end{cases} \quad (5)$$

where k is an arbitrary continuation. Treating k as arbitrary is decisive. A candidate step is not allowed to exploit the behavior of one guessed tail function that happens to occur in a few examples.

Theorem 2.1 (Fold contract). *If b and a satisfy (4) and (5), then for every finite list xs and integer i ,*

$$\text{foldr}(a, b, xs)(i) = J_d(xs, i).$$

Proof. For the empty list, the left side is $b(i)$, so the base contract proves the claim. Suppose the claim holds for xs at every integer index. The fold of $h :: xs$ is $a(h, \text{foldr}(a, b, xs))$. For a negative index and for zero, the step contract gives the first two branches of (3). For a positive index it gives $\text{foldr}(a, b, xs)(i - 1)$, which is $J_d(xs, i - 1)$ by the induction hypothesis. These cases exhaust the integers. The quantification over every index in the induction hypothesis is exactly why the carrier is $\mathbb{Z} \rightarrow A$. $\square$

The synthesized term has the following shape:

```
fun default outerIndex input =>
  input (Int -> A)
  (fun head tailFn localIndex =>
    if localIndex < 0 then default
    else if localIndex = 0 then head
    else tailFn (localIndex - 1))
  (fun _ => default)
  outerIndex
```

This is explanatory Lean-like syntax. The archive's executable version uses a small explicitly typed intermediate language, and its Lean specimen states the corresponding theorem for encoded lists.

2.4 The Church representation boundary

For a finite list xs , define its Church encoding by

$$\text{encode}(xs)(R, a, b) = \text{foldr}(a, b, xs).$$

Theorem 2.1 immediately proves the indexing theorem for $\text{encode}(xs)$. It does *not* establish that every arbitrary Lean value with a Church-style type behaves as the encoding of a finite list. That stronger statement requires an appropriate representation or naturality hypothesis and a proof in the actual source theory.

This matters especially in a synthesis engine that treats polymorphic values as opaque data. A source type is not an automatic relational-parametricity theorem. The prototype’s acceptance language is deliberately named `encoded-finite-list-index-v1`. Changing it to an unrestricted Church-value claim causes the independent checker to reject the certificate. The article’s theorem and the executable checker therefore have the same scope.

2.5 The executable search

The prototype builds the fold sketch, but does not insert the answer’s four expressions as already chosen leaves. It enumerates four finite hole grammars: the base result, the negative branch, the zero branch, and the positive branch. The context contains the default, the input fold, the outer index, and n unhelpful element-valued binders. The step additionally contains the head, continuation, and local index. Candidate continuation arguments include $i + 2, i + 1, i, i - 2, i - 3, i - 1$ in that order.

The flat policy enumerates tuples from the four grammars, type-checks each whole term, and evaluates the same six training examples. The local policy first filters each hole against its local contract, using distinct symbolic-looking values for element binders and a continuation observation $k(j) = (\text{tail}, j)$; it then enumerates the surviving tuples. Both policies use the same grammar and candidate order within each hole.

Those local observations remain a search filter. Final acceptance uses an independent symbolic checker: it checks the carrier, the base and step binders, the outer application, the three branch results, and the affine predecessor argument. In particular, the positive branch must call the locally bound continuation at precisely $i - 1$, not at the outer index. This checker is narrow by design, but its success is stronger than passing a finite example suite. The proof that the checked algebra implies list correctness is Theorem 2.1; its Lean formalization is a remaining integration gate.

```
completeFold(demand, specification):
  for spine in compatibleEliminationSpines(demand):
    R := carrierEquation(spine)
    sketch := instantiateFoldAt(R)
    contracts := deriveBaseAndStepContracts(specification, sketch)
    for compatibleTuple in join(localSearch(contracts)):
      term := assemble(sketch, compatibleTuple)
      if sourceTypeCheck(term) and proveAlgebraContracts(term):
        return applyCheckedFoldTheorem(term)
  return Unknown
```

The implemented fragment has one selected fold schema, one carrier spine, and a fixed three-way integer case split. Automatic discovery of arbitrary recursive schemas, branch predicates, and source-level contracts is proposed rather than implemented. The value of the prototype is to isolate and measure local composition once these structures are available.

2.6 Do not replace relational compatibility by projections

Independent filtering is exact only when the specification factors into independent local conditions. Suppose two holes u, v must satisfy $u + v = 1$ and their domains are $\{0, 1\}$. The allowed relation is

$$C = \{(0, 1), (1, 0)\}.$$

Projecting onto the two coordinates and taking their Cartesian product introduces $(0, 0)$ and $(1, 1)$. Both projections are individually correct, but their product is not the original relation.

A general implementation should therefore store residual constraints as finite relations or symbolic relations indexed by *shared hole identifiers*. For finite relations, use natural joins: if R_1 mentions holes (u, v) and R_2 mentions (v, w) , index both by the full identity of v , join matching rows, and retain witnesses for all constraints. For symbolic relations, keep shared metavariables owned by the same synthesis region and discharge the conjoined residual condition before accepting an assembled term.

The indexing fragment factors after the constructor contract is fixed: the base, negative, zero, and positive results have separate conditions. Its one-survivor-per-hole result must not be generalized into a claim that all higher-order synthesis admits such a product decomposition.

2.7 Source identity, instantiation, and scheduling

The proposed Leant/Djex integration needs one new search object: an *algebra region*. It retains the exact fold occurrence, selected source polytype, instantiated carrier, source universes, available lexical binders, local instance evidence, hole dependency relation, and residual contract. These are references to the existing typed-source inventory, not fresh spellings that will later be guessed back into the original context.

Each hole receives only the source binders its contract permits. Removing a binder from a search context is a completeness-affecting choice unless a separation argument justifies the smaller interface. Accordingly, the first implementation should use a smaller interface as a preferred search branch, while preserving a budgeted branch with the original context. The finite prototype instead fixes the interface explicitly as part of its problem.

Search continuations should be memoized by the algebra region and hole, not by pretty-printed type alone. A useful memo entry contains the next candidate cursor, already considered local terms, remaining shared budget, and the residual compatibility relation. Sibling holes may reuse a function head even when an ancestor-path rule forbids repeating that same head along a single application path; that distinction already matters in Djex’s construction policies [9].

This is not an invitation to reset the old global limits. Local checks, joins, whole-term construction, and verification must all be charged. The intended advantage is fewer useless complete combinations, not more hidden computation. A cve5 SyGuS worker could later solve first-order arithmetic holes inside a fixed carrier, but it should not be asked to recover Lean’s complete dependent or rank- N source semantics [23]. The initial implementation needs no SMT solver.

2.8 What the experiment isolates

With 12 distractor binders, the flat search reaches its first accepted term after 50,940 whole-term candidates. The local search checks 61 components and assembles one whole term. A 256-candidate cutoff truncates the flat run, whereas the local run succeeds. These numbers measure the supplied finite grammar, whose ordering puts distractors early for both policies. They are not a measured 50,940-fold speedup and not evidence that the original Leant query has been fixed.

The useful experimental conclusion is narrower and stronger: an explicit constructor contract can remove the combinatorial cross-product that causes a composition search to repeatedly combine locally wrong

pieces. The production test must preserve Leant's original source signature, primitive inventory, binder order, behavioral predicate, and limits; the easier isolated step is a regression fixture, not a substitute acceptance target.

3 Coupled invariants by descending linear spaces

3.1 Conservation is stronger than induction needs

An equation $p(x) = 0$ can be invariant even if $p(F(x)) \neq p(x)$ as a polynomial. For example, $p(F(x)) = 2p(x)$ preserves zero. More generally, a family of equations can preserve its common zero set while its individual components mix:

$$\mathbf{p}(F_a(x)) = M_a \mathbf{p}(x), \quad \mathbf{p} = (p_1, \dots, p_r)^\top, \quad M_a \in \mathbb{Q}^{r \times r}. \quad (6)$$

This is a genuine step beyond searching only for conserved quantities.

Forge explicitly recognizes the still more general relation $p_i(F(x)) = \sum_j h_{ij}(x) p_j(x)$ and correctly warns that jointly discovering unknown p_j and unknown multipliers is bilinear [2]. The algorithm in this section does not pretend otherwise. It restricts the multipliers to constants and computes a greatest stable subspace by linear preimages, never solving for a new invariant basis and its action matrix in one bilinear system. Section 4 removes the constant-multiplier restriction by a different, target-seeded algorithm.

3.2 Problem statement

Fix finitely many polynomial maps $F_1, \dots, F_s : \mathbb{Q}^n \rightarrow \mathbb{Q}^n$ and a polynomial initialization $\phi : \mathbb{Q}^k \rightarrow \mathbb{Q}^n$. Every finite word in the maps is allowed; thus the problem includes arbitrary switching between transitions. The desired target is an equation $q(x) = 0$ along every resulting orbit.

Let $P_d = \mathbb{Q}[x_1, \dots, x_n]_{\leq d}$. Its dimension is

$$N = \binom{n+d}{d}.$$

The initialization induces a linear map $E_\phi : P_d \rightarrow \mathbb{Q}[t_1, \dots, t_k]$, $p \mapsto p \circ \phi$. Each transition induces a linear pullback map $F_a^* : p \mapsto p \circ F_a$. Pullback is linear in the polynomial being transformed, even when F_a itself is nonlinear. Its image need not lie in P_d .

Define

$$V_0 = \ker E_\phi, \quad (7)$$

$$V_{j+1} = V_j \cap \bigcap_{a=1}^s (F_a^*)^{-1}(V_j). \quad (8)$$

All spaces on the right are subspaces of P_d . Membership of a pullback in V_j is tested as an equality of full polynomials, not after projection back to degree d .

Theorem 3.1 (Greatest stable bounded-degree space). *Iteration (8) stabilizes after at most N strict dimension decreases. Its fixed point V_* is the greatest subspace of P_d whose polynomials vanish on $\phi(\mathbb{Q}^k)$ and which is stable under every F_a^* . Every polynomial in V_* vanishes on every reachable state.*

Proof. The sequence is descending in the finite-dimensional space P_d . A proper inclusion decreases dimension by at least one; once equality occurs, applying the same operator again leaves the space unchanged.

At a fixed point, $V_* \subseteq V_0$ and $F_a^*(V_*) \subseteq V_*$ for every a . Let $W \subseteq P_d$ have those two properties. Then $W \subseteq V_0$. If $W \subseteq V_j$, stability gives $F_a^*W \subseteq W \subseteq V_j$, so $W \subseteq V_{j+1}$. Hence $W \subseteq V_*$.

Finally, all elements of V_* vanish initially. If they all vanish at x , then for $p \in V_*$, the polynomial $p \circ F_a$ also belongs to V_* , so $p(F_a(x)) = 0$. Induction on the length of an arbitrary transition word proves the reachability assertion. $\square$

3.3 An executable preimage computation

Choose an ordered monomial basis for P_d and keep V_j in reduced row-echelon form, with row basis $b_1, \dots, b_r$. For each a and i , expand $b_i \circ F_a$ exactly. Eliminate its coefficients at the pivot monomials of V_j using the basis rows, retaining all remaining monomials. Denote the remainder by $u_{a,i}$.

A combination $p = \sum_i c_i b_i$ has its pullback in V_j exactly when

$$\sum_i c_i u_{a,i} = 0 \quad \text{for each } a.$$

Stack the coefficient equations for all remainder monomials and transitions. Their nullspace gives the allowed vectors c . Multiply that nullspace basis by the old row basis and row-reduce once more. This computes (8) with ordinary exact linear algebra.

```
V := nullspace(coefficients(monomialBasis o initialization))
repeat:
  residuals := []
  for transition F:
    for basis polynomial b in V:
      residuals.append(fullRemainder(b o F, vectorSpace = V))
  C := simultaneousCoefficientNullspace(residuals)
  next := canonicalRowSpace(C * V)
  if next = V: break
  V := next
return V, exactActionMatrices(V), targetCoordinates(V)
```

The phrase *full remainder* is load-bearing. If $x' = x^2$ and $d = 1$, then $x \circ F = x^2$ is not a degree-one polynomial. Replacing it by zero would falsely make $\text{span}\{x\}$ look stable. The prototype retains the x^2 coefficient and rejects this proposed stability. Failure to find a stable space is a search limitation, not a proof that $x = 0$ fails on the orbit.

3.4 A coupled nonlinear example

Take the parameterized initial state

$$(x, y, z) = (t, t^2, 2t^2), \quad t \in \mathbb{Q},$$

and allow either transition

$$\begin{aligned} F_1(x, y, z) &= (2x, 2y + z, y + 3z + x^2), \\ F_2(x, y, z) &= (-x, 2x^2 - y, y + z - x^2). \end{aligned}$$

We want to prove $z = 2y$ after every finite sequence of these updates.

Set $e_1 = y - x^2$ and $e_2 = z - 2x^2$. Direct expansion gives

$$\begin{pmatrix} e_1 \circ F_1 \\ e_2 \circ F_1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \end{pmatrix}, \quad \begin{pmatrix} e_1 \circ F_2 \\ e_2 \circ F_2 \end{pmatrix} = \begin{pmatrix} -1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \end{pmatrix}.$$

The target is $e_2 - 2e_1$. Neither equation needs to remain numerically unchanged outside its zero set; only the pair's common zero set needs to be preserved.

The degree-two search has 10 monomials. Initialization leaves a five-dimensional vanishing space, and one strict refinement reduces it to the two-dimensional stable space generated by e_1, e_2 . The stored canonical basis happens to use the order (e_2, e_1) ; its action matrices are correspondingly permuted. The conservation-only ablation, with the same degree and initialization, has dimension zero. A degree-one search also has zero stable dimension and misses the target.

These are exact nullspace calculations, not fits to sampled trajectories. Initialization is checked after substituting the symbolic parameter t , and each step relation is checked coefficient by coefficient.

3.5 A stronger completeness statement for affine transitions

For nonlinear transitions, V_* may omit valid degree- d invariants because preserving them can require higher-degree multiples. If every F_a is affine, then P_d is itself pullback-stable, and a sharper statement holds.

Theorem 3.2 (Affine-fragment completeness). *For affine transitions and polynomial initialization over $\mathbb{Q}$, V_* is exactly the set of degree-at-most- d polynomials that vanish on all reachable states.*

Proof. Let W be that set. It is a subspace of P_d and vanishes initially. If $p \in W$, then $p \circ F_a$ has degree at most d . For any reachable x , $F_a(x)$ is also reachable because every transition is enabled, so $(p \circ F_a)(x) = 0$. Thus W is pullback-stable and is contained in V_* by Theorem 3.1. The reverse inclusion is its soundness conclusion. $\square$

The prototype tests this theorem by a second, forward formulation. Let $m(x)$ be the column of all monomials through degree d . For an affine map, $m(F_a(x)) = L_a m(x)$ for an exact rational matrix L_a . Start from the span of coefficient vectors of the parameterized feature column $m(\phi(t))$, and close that span under all L_a . Its annihilator is the space of polynomial relations.

Why use coefficient vectors rather than only a few initial feature values? A rational linear functional vanishes on all evaluations of a polynomial vector if and only if the resulting polynomial is identically zero, which over the infinite field $\mathbb{Q}$ is equivalent to every coefficient vanishing. Thus the two spans have the same annihilator. Forward reachable-span closure and backward stable-space refinement must therefore agree.

They agree on all 72 generated affine test systems. Of these, 24 have a nonzero relation space; the others are useful zero-space controls, not additional nontrivial theorems. The two algorithms share the exact scalar-arithmetic helper, so this is algorithmic differential testing, not two wholly independent implementations of rational arithmetic.

3.6 The certificate and its proof cost

A positive certificate contains the polynomials p_i , one constant matrix M_a per transition, and target coordinates $q = \sum_i c_i p_i$. The checker verifies

$$p_i \circ \phi = 0, \quad p_i \circ F_a = \sum_j (M_a)_{ij} p_j, \quad q = \sum_i c_i p_i. \quad (9)$$

It neither recomputes the fixed point nor trusts the searcher's claim of maximality. Linear independence of the listed polynomials is also unnecessary for certificate soundness; redundant generators merely cost space.

A Lean implementation can express the invariant as a finite conjunction of polynomial equations and use a single generic reachability-induction lemma. For small certificates, generating direct expressions and discharging the identities with `ring` or `ring_nf` is the simplest bridge. For larger ones, a reflected sparse-polynomial checker should use an established polynomial semantics such as `MvPolynomial.eval2`, whose definition and evaluation lemmas are present at the pinned Mathlib revision [18].

The expensive work is not the theorem above but coefficient growth and substitution. At degree d , a transition of degree D can produce monomials through degree dD , with ambient count $\binom{n+dD}{n}$. There are at most N strict refinements, but this bound does not make the arithmetic polynomial in the input bit length or keep the intermediate polynomials small. Cache monomial pullbacks, exploit sparse rows, and use fraction-free or modular exact linear algebra in production. Python-FLINT supplies rational row reduction and integer nullspace facilities that can serve the proposer; acceptance still uses the exact identities [19, 20].

3.7 Guards and control locations

Ignoring guards enlarges the transition relation. An invariant proved for the enlarged system is valid for the guarded system, but failure in the enlarged system says nothing negative about the guarded source program. Parameters in ϕ are likewise universally unconstrained in this fragment.

A useful unimplemented extension is one space V_ℓ per control location. For an edge $e : \ell \rightarrow m$, require $F_e^* V_m \subseteq V_\ell$, with separate initial constraints at entry locations. Simultaneous descending refinement in $\prod_\ell P_d$ still terminates after at most $|L|N$ strict dimension drops. The certificate becomes a family of rectangular action matrices, one per edge. This is the natural representation for mutually recursive helpers whose invariants differ by phase; it is preferable to artificially embedding every control location in the same polynomial equation list.

4 Target-seeded inductive ideal completion

4.1 The missing case after linear-space closure

Consider $x' = x^2$, initial state $x = 0$, and target $x = 0$. The degree-one stable-space algorithm correctly fails to keep $\text{span}\{x\}$: its pullback is x^2 , outside that vector space. But the ideal $\langle x \rangle$ is stable, because $x^2 = x \cdot x$. A polynomial multiplier is enough to prove the target.

This suggests a second lane. Instead of guessing a complete invariant basis and its multipliers simultaneously, start with the desired target and close the *ideal it generates* under pullback. The auxiliary equations are then forced by failed preservation checks. This is a target-directed specialization of established algebraic invariant-set methods, not a new claim about the foundations of invariant generation. In particular, the recent treatment by Bayarmagnai, Mohammadi, and Prébet studies invariant-set computation and polynomial-invariant testing with stronger geometric stopping criteria [14]. Here ordinary ideal membership is chosen because it returns especially simple proof objects for Lean.

4.2 A completion algorithm

For the same unguarded rational-polynomial model as Section 3, let q be the target polynomial. Form the ascending sequence of ideals

$$J_0 = \langle q \rangle, \quad J_{j+1} = J_j + \sum_{a=1}^s \langle F_a^* J_j \rangle. \quad (10)$$

The notation on the right denotes the ideal generated by all pullbacks of members of J_j . If G_j generates J_j , it suffices to use the pullbacks of those finitely many generators, because pullback is a ring homomorphism:

$$\left(\sum_i h_i g_i \right) \circ F_a = \sum_i (h_i \circ F_a)(g_i \circ F_a).$$

The implementation adds one nonmember pullback at a time rather than computing all of (10) eagerly. It keeps generators in their original unreduced form, together with the transition word that produced them from q . A Gröbner basis is an internal membership index, not the authoritative list of invariant equations.

```

G := [(target, emptyWord)]
repeat:
  for (g, word) in G:
    if g o initialization is not the zero polynomial:
      parameters := exactNonzeroEvaluation(g o initialization)
      return OrbitCounterexample(parameters, word)
  B, changeMatrix := tracedGroebnerBasis(polynomials(G))
  for transition F_a and (g, word) in G:
    p := g o F_a
    if p is not in ideal(G):
      append (p, prepend(a, word)) to G
      restart
  return initial-zero, pullback-membership, and target certificates

```

Prepending the new transition label is intentional. If $g(x) = q(F_{a_m}(\dots F_{a_1}(x)))$, then $g \circ F_b$ corresponds to executing b first and then $a_1, \dots, a_m$. A noncommuting-transition regression test checks this orientation independently by executing the returned word.

Theorem 4.1 (Target completion for the unguarded model). *With exact ideal membership and without resource cutoffs, the algorithm terminates. It either proves that q vanishes on every reachable state from $\phi(\mathbb{Q}^k)$, or returns a rational parameter tuple and a finite transition word on which q is nonzero.*

Proof. Every added polynomial lies outside the current ideal, so each addition strictly increases that ideal. The polynomial ring $\mathbb{Q}[x_1, \dots, x_n]$ is Noetherian; there can be only finitely many strict increases. Exact ideal membership is effective by Gröbner-basis reduction. Thus either an initialization failure is found or a pullback-stable ideal is reached.

At stable completion, all generators vanish initially, each generator's pullback belongs to their ideal, and q belongs to that ideal. Evaluating a polynomial combination at a common zero produces zero. Induction on a transition word therefore proves the target.

For the other outcome, every retained generator is an actual pullback of q along its recorded word, not just a polynomial inferred by an untracked basis change. If $g \circ \phi$ is nonzero, there exists a rational parameter tuple where it evaluates nonzero. Concretely, if its degree in each parameter is at most D , some point in $\{0, \dots, D\}^k$ is nonzero: induction on the number of variables reduces this fact to the bound on roots of a nonzero univariate polynomial. Executing the recorded word at that parameterized initial state therefore yields a counterexample to the target. $\square$

This is a decision result for a *specified polynomial equality* under unconditional polynomial updates and unrestricted rational initialization parameters. It is not a procedure for computing the full strongest invariant ideal of an arbitrary program. Those are different problems; the known hardness results for strongest polynomial invariants do not justify conflating them [15].

4.3 Why completion avoids the bilinear synthesis trap

The two unknowns in a general preservation equation are the invariant family and its polynomial multipliers. Completion never chooses both freely in one constraint solve. At a membership check, the current generators are fixed; the problem is to express a particular pullback in their ideal. If that fails, the pullback itself becomes a new generator. All added equations are dictated by the target and transition maps.

This has a useful proof-theoretic interpretation. A failed induction step asks for precisely the missing equation needed to make that step close. Completion accumulates those demands until they are mutually inductive. It is a specialized, fully algebraic form of auxiliary-lemma discovery, rather than a language-model proposal of unrelated lemmas.

For the coupled example, starting from $q = z - 2y$ under F_1 produces

$$q \circ F_1 = z - 3y + x^2.$$

This new equation is not an ideal multiple of q and is added. The two equations span the same linear family as e_1, e_2 from the earlier example, and both transitions subsequently close. The implementation needs two generators and one successful enlargement. Unlike the vector-space lane, it did not begin by constructing every degree-two polynomial that vanishes initially.

4.4 A genuinely polynomial action

Take initialization $(x, y) = (t^2, t^3)$ and transition $(x, y) \mapsto (x^2, y^2)$. The target $g = y^2 - x^3$ satisfies

$$g \circ F = y^4 - x^6 = (y^2 + x^3)g.$$

The target has degree three, but its pullback has degree six. A degree-three vector-space closure cannot retain g merely by ignoring those higher-degree terms. The ideal lane needs just one generator and the explicit multiplier $y^2 + x^3$.

More generally, $u^a - v^a = (u - v) \sum_{j=0}^{a-1} u^{a-1-j} v^j$ gives a family of nonconstant action certificates for power-curve invariants. Twelve such fixtures are included. They are constructed examples with known invariants, useful for checking polynomial multiplication and substitution; they are not a held-out benchmark of discovery performance.

4.5 Traced membership, not a trusted CAS answer

The search-side Gröbner implementation carries a representation

$$b_j = \sum_i T_{ji} g_i$$

for every intermediate basis element. Forming an S -polynomial, reducing it, and making it monic applies the same polynomial operations to its representation row. If division gives $p = \sum_j u_j b_j$ with zero remainder, the desired original-generator coefficients are $h_i = \sum_j u_j T_{ji}$.

Only the final identities are sent to the checker:

$$g_i \circ \phi = 0, \quad g_i \circ F_a = \sum_j h_{aij}(x) g_j(x), \quad q = \sum_i c_i(x) g_i(x). \quad (11)$$

The checker does not need to verify a Gröbner basis, rerun Buchberger, or trust a CAS's zero-remainder flag. Conversely, a zero remainder relative to a new basis is insufficient unless the change-of-generators expressions are retained.

For a production proposer, Singular’s `lift` and `liftstd` facilities are concrete candidates for obtaining these representations; their output must still be converted to the original generator order and checked [21]. SymPy is a convenient differential oracle for small cases, but its ordinary basis reduction interface should not be mistaken for an already integrated source-to-Lean certificate adapter [22].

4.6 Counterexamples and conservative refusal

The negative certificate is deliberately smaller than the positive one: it contains initialization parameters and a finite word of transition indices. The independent checker evaluates initialization, executes the transitions in order using exact rationals, and verifies that the externally supplied target is nonzero. It never trusts the intermediate polynomial that led the proposer to that word.

Three fixtures produce such certificates: one fails after one step, another after two steps, and a third requires the order $[1, 0]$ of two noncommuting maps. Reversing that last word is rejected. An ideal-generator cutoff separately produces `unknown`, not a counterexample or an unprovability judgment.

When guards have been erased, an orbit counterexample is only a counterexample to the enlarged model. To refute a source-level theorem, the adapter must replay the same trajectory through the source guards and initialization domain. A negative guard check invalidates the source-level counterexample, even though the unguarded encoded observation remains correct.

4.7 Where this lane belongs in the scheduler

Noetherian termination is not a practical complexity guarantee. Gröbner bases, coefficient sizes, and pullback degrees may become prohibitive. The prototype limits generator count, pullback degree, division steps, and basis pairs; a production worker additionally needs process-level memory and time limits, because an expansion can become expensive before its completed degree is inspected.

Run the linear-space lane first when $\binom{n+d}{d}$ is small and several related targets are expected. Its result is reusable across all targets in the computed space. Run ideal completion when a specific failed equality is available, especially after the space lane reports degree escape. The latter is a useful *routing signal*, not a reason to truncate the polynomial representation.

The ideal lane is also a bridge to the cyclic compiler: a discovered family of equations can serve as the simultaneous induction motive for a recursive arithmetic helper. The two backends then cooperate through explicit local proof obligations rather than through an unstructured bag of candidate equations.

5 Cyclic obligations compiled to well-founded proofs

5.1 A repeated goal is a candidate recursive call

Existing tabling must not turn an active goal into its own proof. The proposed extension preserves that rule. It changes what happens *after* a recursive pattern has been found: a cyclic dependency may be recorded as an unaccepted proof-program candidate, whose recursive calls must subsequently be justified by well-founded descent.

This direction is motivated by cyclic equational proof systems such as CycleQ, and by recent work translating abstract cyclic proofs into induction [16, 17]. The first Forge implementation should be substantially more restricted than those general results. It should require a concrete scalar or lexicographic ranking that Lean can use with ordinary well-founded induction. There is no need to trust an arbitrary cyclic proof graph or implement a general cyclic-proof kernel.

5.2 The typed object to compile

Each graph node v has a telescope Γ_v , a state type S_v packaging its varying inputs, and a proposition family $P_v : S_v \rightarrow \text{Prop}$. An edge $e : v \rightarrow w$ carries a typed argument map $\sigma_e : S_v \rightarrow S_w$ together with any branch condition under which that call is needed. A local constructor must prove $P_v(x)$ from the recursive calls actually used in that branch and any already checked nonrecursive facts.

These are logical edges, not just syntactic similarities. A matched variable permutation needs a well-typed substitution; a cast between different index types needs a proof; and a change to a dependent hypothesis needs an explicit transport. The tagged state space is

$$S = \sum_{v:V} S_v, \quad P(v, x) = P_v(x).$$

The tag allows mutually recursive lemmas with different input types to be compiled as one induction over a sum type. In an initial prototype integration, a homogeneous telescope can simplify implementation, but it should not be baked into the mathematical contract.

Theorem 5.1 (Ranked local constructors). *Let $\prec$ be a well-founded relation on S . Suppose that for every $s : S$, a checked local constructor proves $P(s)$ from proofs of $P(t)$ for the recursive calls it uses, and each such call has a checked proof $t \prec s$. Then $\forall s, P(s)$.*

Proof. Apply well-founded induction to P . At state s , the induction hypothesis supplies $P(t)$ for each $t \prec s$. The checked descent proofs permit every recursive call in the local constructor to be replaced by an instance of that hypothesis. Applying the constructor proves $P(s)$. $\square$

The theorem requires all branch cases to be covered and all local constructors to be checked. A ranking of a graph alone does not meet those requirements. The included Python code checks only the descent component for supplied graphs; it does not extract those graphs from Lean or construct their logical branches.

5.3 Why node phases matter

Consider two obligations with calls

$$P(n) \longrightarrow Q(n), \quad Q(n) \longrightarrow P(n-1) \quad (n \geq 1).$$

The first edge does not decrease n . Rejecting it independently would miss a valid mutual-induction structure. A rank on the tagged state resolves this:

$$\rho(P, n) = 2n + 1, \quad \rho(Q, n) = 2n.$$

Each edge now decreases by exactly one. The phase is not an arbitrary tie-breaker in search order: it is part of the mathematical termination measure.

The prototype synthesizes this rank by enumerating small nonnegative weights and node phases. A production solver can instead synthesize phases jointly with affine weights. An acyclic run of zero-progress edges suggests phases cheaply: assign reverse topological heights within a candidate phase graph, then scale the primary measure sufficiently to dominate phase resets. Cycles of purely zero-progress edges cannot be repaired by phases alone.

5.4 Permuted arguments and lexicographic growth

Two other fixtures distinguish useful ranking families.

For a call $(x, y) \mapsto (y, x - 1)$ under $x \geq 1$, neither a fixed-position “first argument always decreases” check nor a requirement that both arguments shrink is appropriate. The rank $x + y$ decreases by one. This also covers the standard subtractive Euclidean branches, when their positive-subtraction guards are retained.

For calls

$$(x, y) \mapsto (x - 1, 2y) \quad (x \geq 1), \quad (x, y) \mapsto (x, y - 1) \quad (y \geq 1),$$

lexicographic rank (x, y) works. There is no globally decreasing nonnegative affine scalar $ax + by + c$ for both branches: the second requires $b > 0$, while the first would require $a - by \geq 1$ for unbounded y . This gives a meaningful ablation between scalar and lexicographic ranking search.

The compiler must use lexicographic well-foundedness, not encode (x, y) as $ax + by$ with a guessed fixed multiplier. Such an encoding is valid only when a bound on the lower-priority component is available and proved.

5.5 A tempting but invalid local rule

Suppose the graph contains

$$(x, y) \mapsto (x - 1, y + 1) \quad (x \geq 1), \quad (x, y) \mapsto (x + 1, y - 1) \quad (y \geq 1).$$

Every edge decreases *some* argument. Nevertheless,

$$(1, 0) \longrightarrow (0, 1) \longrightarrow (1, 0)$$

is an infinite repeatable cycle. The decrease on one edge is replenished by the other. Neither a local decreasing-variable flag nor the existence of one favorable cycle proves global well-foundedness.

The prototype’s ranking search returns *unknown* here. Its size-change closure also fails the acceptance criterion. The concrete two-cycle explains this particular failure, but exhaustion of a ranking grammar in an arbitrary case is not itself evidence of nontermination.

5.6 Size-change closure as a proposal filter

A size-change graph records relations between caller measures and callee measures. The implementation uses entries 0, 1, 2 for unknown, nonincrease, and strict decrease. Along a composable path, an unknown comparison annihilates that path; otherwise it is strict if either constituent is strict. Multiple paths between an input and output retain the strongest established comparison.

Writing $\otimes$ for path composition and $\oplus$ for maximum strength,

$$(G \circ H)_{ij} = \bigoplus_k G_{ik} \otimes H_{kj}.$$

Compute the closure of graphs of *nonempty* paths, retaining source and target node tags. The standard size-change test requires a strict diagonal entry in every idempotent self-graph in that closure. A failure means that this abstraction did not certify descent; a success relies on all recorded local comparisons being semantically valid [16, 17].

In this implementation the closure is diagnostic and guides ranking search. It is not passed off as a Lean termination proof. The first compiler accepts only an explicit checked ranking. A later release could formalize an abstract size-change soundness theorem or implement a general unravelling construction, but that is separate work with a separate proof obligation.

5.7 Exact affine and lexicographic certificates

The executable graph language uses natural-number measures and guarded affine updates represented over the integers. For an edge $e : v \rightarrow w$ with update F_e , a scalar ranking certificate proves

$$\rho_v(x) - \rho_w(F_e(x)) - 1 \geq 0.$$

Subtracting one encodes strict decrease on integral ranks. Replacing this condition by nonincrease is a soundness bug, not a harmless relaxation.

The checker proves the inequality by an explicit nonnegative rational combination of the edge’s guard polynomials, the natural-domain inequalities $x_i \geq 0$, and the constant inequality $1 \geq 0$. It separately proves that every updated measure is nonnegative. Without the latter check, a syntactically decreasing integer expression could leave the claimed well-founded domain.

For a lexicographic tuple, the certificate gives a pivot for every edge. All components before the pivot must be equal, and the pivot decreases by at least one. Later components need not decrease. The implemented checker requires literal affine equality for the earlier components; a production extension may accept guarded equality proved by two checked inequalities instead.

All ranking coefficients and phases in the prototype are nonnegative integers, so the ranks map every natural state to a natural. More expressive ranks can have signed coefficients, but then their nonnegativity over each node’s invariant region becomes an additional obligation. They should not be admitted merely because they decrease along the sampled calls.

5.8 A concrete production synthesis algorithm

For one affine rank, use unknowns w_{vi} and phases b_v in $\rho_v(x) = b_v + \sum_i w_{vi}x_i$. For each edge, introduce nonnegative Farkas multipliers λ_{ej} and equate coefficients in

$$\rho_v(x) - \rho_w(F_e(x)) - 1 = \sum_j \lambda_{ej} g_{ej}(x).$$

The update and guard coefficients are fixed, so this is a linear feasibility problem in the rank parameters and multipliers. Solve over rationals, scale to integral rank coefficients if necessary, regenerate all exact identities, and check them. The solver’s numerical feasibility verdict is not the certificate.

For a lexicographic tuple of fixed length, choose a pivot for each edge, enforce prefix equality, and solve the resulting linear problem. Pivot choices can be enumerated for small SCCs or represented by finite selectors in an SMT/MILP proposer. The backend must export the selected pivots and exact coefficient witnesses, not a floating-point tuple that is rounded without rechecking.

The Python implementation uses bounded enumeration instead: common weights across nodes, small individual phases, then permutations of the original coordinates as a lexicographic fallback. Cone membership is solved by enumerating linearly independent supports of size at most the affine coefficient dimension. This is appropriate for its one- and two-variable tests; it is not proposed as a scalable replacement for an exact LP backend.

5.9 Compiling an SCC without introducing circular declarations

A practical Lean compiler can close one strongly connected component as follows. First, package its node goals into the tagged family P . Second, construct a well-founded relation from the accepted rank. Third, build one local proof constructor per node using ordinary case splits and the already checked nonrecursive facts. Recursive edges become accesses to the well-founded induction hypothesis, instantiated at the typed

callee state and supplied with the corresponding descent proof. Finally, project the theorem for the entry node and original arguments.

No node theorem should be inserted into the environment as an assumed theorem while its SCC is being checked. The only recursive authority is the induction hypothesis provided by the well-founded recursor. Lean supports structural and well-founded recursive definitions through ordinary kernel-checked terms; `termination_by` and `decreasing_by` are possible elaboration interfaces, not a new trusted proof rule [24, 25].

The first implementation should support explicit tagged states and *given* local constructors, then add graph extraction. Trying to discover dependent back-edge substitutions, generate branches, infer ranks, and lower proofs all at once makes failures difficult to localize. The supplied examples provide separate descent gates before that larger integration begins.

6 Putting the workers into Forge

6.1 Three new payloads, not a new authority model

Forge’s proposed runtime already distinguishes an ordinary candidate from a conditional candidate whose premises still need proofs. The extension should reuse that distinction and the existing acceptance boundary, rather than create a second notion of accepted proof [5, 6]. The new data is the *shape of the candidate*:

```
AlgebraRegion:
  sourceFoldOccurrence
  selectedCarrierAndInstantiation
  localHoleTelescopes
  residualContracts
  sharedHoleConstraints
  reconstructionTerm

InductiveAlgebraCandidate:
  stateSchema
  initialSubstitution
  labeledTransitions
  equations
  actionMatricesOrPolynomialMultipliers
  targetCombination

RankedSCCCandidate:
  nodeTelescopes
  propositionFamilies
  branchProofConstructors
  typedCallSubstitutions
  rankAndDescentCertificates
```

These are proposed records. They are not currently wired into `ForgeDesign.EngineReply`, and their fields do not themselves enforce soundness. Their advantage is that every unresolved field corresponds to a specific next action: prove a local contract, verify a polynomial identity, transport a dependent argument, or discharge a descent condition.

6.2 A target-directed worker protocol

The polynomial worker receives an already reified transition problem and a specific target equation. It may return a proved inductive family, an exact encoded-model counterexample, a degree-escape report, or

a bounded miss. The first can become a conditional candidate for the original theorem once initialization, transition semantics, and the target transport are supplied. A degree-escape report can trigger the ideal lane; it cannot close the goal.

The continuation worker receives a fold occurrence and a residual specification. It returns a typed term plus algebra-contract obligations. Successful local search does not replace the final source elaboration. A term that fits the neutral carrier but fails the source universe constraints is rejected, even if its Python interpretation works on every test list.

The cyclic worker receives a candidate SCC and returns a ranked proof-program skeleton. It does not emit ordinary checked facts for individual nodes before the whole component has been lowered through well-founded induction. Its local constructors can use algebraic facts from the invariant worker, but only after those facts have been proved in the appropriate node context.

6.3 A cooperative proof trace

Consider a helper that performs either of the two polynomial updates from Section 3, selected by a list of commands, and returns its final state. The desired theorem is that the final state satisfies $z = 2y$ when the initial state is $(t, t^2, 2t^2)$.

A direct induction on commands reaches two arithmetic step goals. Inducting only on the target equation is insufficient: $z = 2y$ alone does not imply that either update preserves it for arbitrary x, y, z . The failed step is exactly the useful signal to send to inductive algebra.

The space lane returns $e_1 = e_2 = 0$ and the two constant action matrices. Alternatively, target-seeded ideal completion begins with $z - 2y$ and adds the missing pullback equation. Forge strengthens the induction motive to the resulting conjunction, proves the initialization by substitution, proves each step by the action identities, and derives the original target as a linear combination. No arbitrary induction predicate needs to be guessed beyond the polynomial fragment.

Now split execution into two mutually recursive phases, with a transition between phases that does not consume a command and a second transition that consumes one. A plain “list length decreases on every call” gate fails. The cyclic worker attaches phase-sensitive rank $2 \cdot \text{length}(xs) + b_v$, then compiles the same strengthened motive through well-founded induction. The action certificates solve arithmetic preservation; the rank certificates solve recursion admissibility. These are different obligations with different proof artifacts.

This combined end-to-end proof is an *integration target*, not an executed benchmark. The archive executes its algebra and descent components separately. That separation is useful: it tells the implementer whether a later failure is in mathematics, reification, dependent transport, or proof construction.

6.4 Dependent contexts require explicit call morphisms

A cyclic edge must map the callee’s entire dependent telescope, not just the visible arguments in its displayed goal. For example, if a node contains $n : \mathbb{N}, xs : \text{Vector}(A, n)$, and a hypothesis about xs , an edge changing n cannot keep the old vector or hypothesis by identity. It must supply a vector at the new index and transport or reconstruct the dependent premise.

Similarly, a proof established under a hypothesis $H(n_0)$ about one fixed current value cannot be silently generalized into a proof under $H(n)$ for all n . The compiler should separate ambient parameters, which remain fixed for the whole SCC, from state-dependent inputs and hypotheses, which are packaged in S_v or explicit premise arguments. Matching a back edge then constructs a morphism of these packages and asks Lean to check it.

This is where a source-aware neutral representation earns its cost. Printed variable names do not identify a telescope, and an alpha-normalized goal body alone does not determine which hypotheses can be moved to

a recursive call. The necessary infrastructure is already part of Forge’s identity design; cyclic compilation adds a precise consumer that requires those identities to be carried through a typed substitution.

6.5 Reification obligations specific to these algorithms

The polynomial algorithms operate over $\mathbb{Q}$ with simultaneous updates. A source adapter must prove that its extracted map has those semantics. Sequential source assignments need an SSA-style substitution pass before they become one map: $x := x + 1; y := x + y$ means $(x, y) \mapsto (x + 1, x + y + 1)$, not $(x + 1, x + y)$. This is an important negative integration fixture.

A natural-number subtraction is not unrestricted ring subtraction. It can be translated into integer subtraction only under the appropriate comparison premise, with that premise retained in the edge’s guard and proved where the edge is used. Division by a variable is not a polynomial update. Floating-point arithmetic is not rational arithmetic. The first adapter should refuse these unproved translations rather than send a nearby polynomial problem to a worker.

For a source recurrence over naturals, proving a polynomial identity after an embedding into a characteristic-zero field can be useful, but the embedding and reflection of the final equality are source-level obligations. A counterexample with rational initialization parameters is not a counterexample to a theorem quantified only over naturals unless those parameters are also shown to lie in the source domain.

6.6 Concrete module boundaries

Proposed module	Responsibility and first acceptance condition
Forge/Synthesis/AlgebraRegion.lean	Extract one source fold occurrence, retain carrier/universe evidence, and construct local telescopes without losing binder identity.
Forge/Synthesis/ContractJoin.lean	Join shared-hole constraints; a correlated-hole fixture must reject incompatible projection products.
Forge/Invariant/StateSchema.lean	Reify a simultaneous rational-polynomial transition with a proved source interpretation. Sequential assignment and natural subtraction are mandatory controls.
Forge/Invariant/PullbackSpace.lean	Consume exact constant-action certificates; the coupled example must produce the original target theorem after kernel replay.
Forge/Invariant/IdealReplay.lean	Consume polynomial multipliers and exact orbit traces without invoking a CAS in the proof checker.
Forge/Recursion/CallGraph.lean	Construct typed node families and call substitutions, initially from explicit annotations.
Forge/Recursion/RankReplay.lean	Prove natural-domain and strict/lexicographic descent, including node phases.
Forge/Recursion/CompileSCC.lean	Lower all local constructors through one well-founded theorem; reject a missing branch even when the graph ranks successfully.

The file names specify new responsibilities, not files claimed to exist in the reviewed repository. A first implementation may combine modules, but it should not combine their acceptance conditions.

Layer	Concrete choice	What still has to be implemented
Exact matrix proposer	Python-FLINT <code>fmpz_mat.rref</code> ; integer nullspaces with <code>fmpz_mat</code>	Preimage assembly, rational reconstruction where used, and action-certificate extraction.
Ideal proposer	Singular <code>lift/liftstd</code> ; traced Buchberger for a minimal prototype	Original-generator provenance, substitutions, resource control, and multiplier serialization.
Small arithmetic holes	cvc5's SyGuS engine, optional	Source-scoped grammar and contract translation; candidate terms still need replay.
Lean polynomial semantics	Mathlib <code>MvPolynomial</code> evaluation and ring normalization	A reifier with interpretation lemmas and certificate-to-proof lowering.
Lean recursive proof	Core well-founded induction and recursive-definition elaboration	Tagged goal families, local proof constructors, source-level call transport.

Table 3: Libraries are components, not finished adapters. References: [19, 20, 21, 23, 18, 24].

6.7 Library choices and substitutions

The executable package does not require these proposer libraries. Its exact search uses the Python standard library; SymPy 1.14.0 was used only for the optional differential test. This keeps the evidence reproducible while making the production replacement points explicit.

6.8 Resource allocation that follows the mathematics

The new workers should consume different budgets. The space worker is naturally bounded by degree, monomial count, coefficient size, and elimination work. The ideal worker needs generator, degree, basis-pair, coefficient, and process-memory budgets. The synthesis worker needs local-candidate, join-row, whole-term, and verification budgets. The cyclic worker needs graph-size, ranking-template, solver, and proof-lowering budgets.

A useful escalation order is deterministic and explainable. Try a small algebra region before enlarging the whole-term synthesis window. Try a low-degree space before invoking ideal completion. Try scalar rankings with phases before lexicographic tuples. Return the reason for escalation in the trace: an escaped pullback degree, a failed local contract, or an SCC requiring more than one ranking coordinate. These reasons are concrete mathematical diagnostics, not an opaque score presented as evidence.

7 Development sequence and release gates

7.1 First release: exact inductive algebra

Implement `StateSchema` for explicit rational-polynomial recurrences and lower the certificate identities to Lean proofs. Start with direct generated ring proofs before investing in reflection. The entry theorem should quantify over arbitrary transition words, not a sampled number of iterations. Replay the coupled example using both the space and ideal certificates, and the power-curve example using nonconstant multipliers.

The release gate is an ordinary theorem at the original target type, with initialization and every transition label accounted for. Record proof construction time, kernel checking time, and proof-term size separately. No successful Python certificate should count toward this gate.

7.2 Second release: one source-preserving algebra region

Add continuation-local synthesis for the specific integer-indexing schema. Preserve the exact source fold, default, integer domain, available primitives, and dependent/universe evidence. Prove the derived local contract, then apply a checked theorem for the source representation.

The original Leant indexing query is the acceptance target. Its isolated step and the finite Python grammar are diagnostic fixtures only. First record the original search branch’s lexical identities, pending goals, and usage history; then compare the same branch with and without region-local scheduling. This isolates whether the new decomposition improves the actual miss rather than merely making a different query easier.

7.3 Third release: explicit cyclic skeletons

Accept a user- or tactic-supplied cyclic skeleton with explicit branch constructors and synthesize only its rank. Prove the two-phase, swapped-argument, and lexicographic-growth examples in Lean. Reject the identity cycle, the alternating replenishment cycle, and a graph with a missing logical base branch.

Only after these pass should Forge infer back edges from recurring obligations. The first automatic mode should be limited to source families with straightforward telescope transport. General dependent cyclic matching is a subsequent capability, not a silent assumption of the initial compiler.

7.4 What would constitute a meaningful improvement

A production evaluation should have separate ablations for carrier selection, local contracts, compatibility joins, invariant-space closure, ideal completion, phase ranks, and lexicographic ranks. Each benchmark should retain its original statement and source context. Measure solved goals under a fixed total budget, not only candidate ordinals or search-only CPU time.

For mathematics-heavy recurrences, the key comparison is target-only induction versus automatically discovered simultaneous invariants. For higher-order construction, it is whole-term enumeration versus locally constrained assembly under the same global accounting. For recursive proofs, it is fixed structural induction versus a successfully lowered ranked SCC. A single aggregate score would hide which new capability actually supplied the proof.

8 Executed experiments and their evidential limits

8.1 Reproducibility envelope

The recorded run used Python 3.13.5 on Linux. The main experiments and standalone replay have no third-party runtime dependency. Their seeds are fixed and their inputs are serialized beside the certificates. The optional algebra comparison used SymPy 1.14.0 with a separate seed. The complete scripts and outputs are in `prototype/` and `results/`.

The main run finished in about 1.8 seconds in this environment. That elapsed time is only a reproducibility observation for these small fixtures. It excludes Lean elaboration, source extraction, solver startup in a production worker, and kernel checking. The algorithmic counts below are more meaningful than ratios of sub-millisecond timings.

Distractors	Flat complete terms	Local checks	Local complete terms	Hole sizes
0	24	13	1	1, 2, 2, 8
4	2,148	29	1	5, 6, 6, 12
8	14,384	45	1	9, 10, 10, 16
12	50,940	61	1	13, 14, 14, 20

Table 4: Same finite grammar and within-hole ordering; different composition strategy. Local checks and complete terms are different work units.

8.2 Continuation composition

All four comparisons return the same term. For the largest context, the term was also checked on all lists of length at most six over each of two two-element alphabets, at ordinary nearby indices and at $\pm 2^{80}$. The independent non-fold reference agrees on all 3,570 checks. Final symbolic algebra replay checks the predecessor expression and lexical structure, rather than relying on those observations as a proof of general behavior.

The distractor ordering is deliberately unfavorable and held fixed across both policies. The experiment demonstrates removal of a cross-product under factored contracts, not robustness under all candidate orders. No native Djex or Leant execution was performed in this study.

8.3 Inductive algebra

Experiment	Recorded result
Coupled nonlinear switching example	Stable degree-two dimensions $5 \rightarrow 2$; target derived; conservation-only dimension 0.
Degree-one ablation	Dimensions $1 \rightarrow 0$; no target derivation.
Affine differential corpus	72 systems; backward-space and forward-feature algorithms agree in every case; 24 have nonzero relation spaces.
Constructed nonlinear family	40 systems; expected targets derived and exact action identities replayed.
Target ideal completion	14 positive fixtures: degree-growth control, coupled target, and 12 power-curve cases.
Exact orbit counterexamples	3 fixtures; every returned parameter/word pair independently executed.
Gröbner differential oracle	32 small systems; 368 polynomial remainders agree with SymPy; 185 traced basis identities independently replayed.

Table 5: Algebra results. Generated families and zero-space controls are identified explicitly rather than reported as a production theorem-proving score.

The Gröbner comparison is important because many positive ideal fixtures need only one generator. Such examples alone exercise polynomial division but barely exercise completion. The separate random two-variable corpus contains two or three quadratic generators and tests both original-generator representations and normal remainders against another implementation.

The high-degree negative control is also meaningful: degree-one linear-space closure does *not* incorrectly prove stability of x under $x' = x^2$. The ideal lane then succeeds legitimately with a polynomial multiplier. This checks the intended handoff between algorithms rather than just two unrelated success lists.

8.4 Descent on supplied cyclic graphs

The SCT closure is a diagnostic over abstract comparison graphs. The recorded rank certificates additionally establish natural-valuedness of callee measures and strict descent, using exact nonnegative combinations.

Fixture	Ranking or outcome	Attempts	SCT closure
Two phases	$2n + 1$ at P , $2n$ at Q	8	5
Swapped arguments	$x + y$	4	3
Subtractive Euclid	$x + y$	4	3
Lexicographic growth	(x, y) lexicographically	16	2
Identity cycle	Unknown; SCT criterion fails	4	1
Alternating decrease	Unknown; SCT criterion fails	17	3

Table 6: The four found rankings pass exact independent descent replay. These are call-graph results, not completed Lean proofs of recursive theorems.

A general failed SCT test or exhausted ranking grammar remains inconclusive; the last two fixtures have explicit bad cycles for independent reasons.

8.5 Independent replay and deliberate corruptions

The standalone replayer accepted 135 encoded certificate records: 113 constant-action invariant records, 14 ideal-preservation records, four ranking records, one indexing-algebra record, and three orbit-counterexample records. An affine record with an empty invariant space and zero target is a valid consistency check, not a nontrivial solved theorem. The count therefore must not be described as “135 Lean theorems proved.”

The replayer contains its own polynomial arithmetic and substitution routines; it imports none of the search modules. It checks identities and inequality combinations rather than rediscovering the candidate. It was also run using `python -S`, which disables normal site initialization and demonstrates that no installed symbolic-algebra package is needed for replay. This remains a Python research checker, not a formally verified implementation or a hardened hostile-input verifier.

Thirty-one named corruptions were rejected. They include wrong action matrices, missing transitions, changed initializations, a fabricated target combination, floating-point and noncanonical rational encodings, omitted call edges, erased phases, wrong lexicographic pivots, missing natural-domain proofs, captured binders, an outer-index escape, a wrong predecessor, changed Church semantics, wrong ideal multipliers, and reversed noncommuting execution order.

The portable regression suite has 29 test methods and passes. Some methods replay the already counted certificates, so these numbers overlap; there is no combined “total benchmark cases” statistic. The optional SymPy comparison is separate from both the portable tests and the acceptance boundary.

8.6 What remains unmeasured

No claim is made here about Mathlib-wide coverage, a new default tactic’s latency, Lean proof-term sizes, kernel replay times, or a solved original Leant behavior cell. The source-level folds, cyclic proof constructors, and polynomial maps in these experiments were supplied or generated in the prototype’s own language. Extracting them from arbitrary Lean goals remains implementation work.

These limitations do not make the experiments merely illustrative. They test specific algorithmic claims: a greatest stable subspace, exact target-driven ideal completion, compositional algebra synthesis, and sound descent certificates. They establish enough of each mechanism to make its next integration gate precise and falsifiable.

9 Conclusion

The most valuable extension to Forge is not a longer list of fallback tactics. It is the ability to discover and certify *interfaces between partial solutions*. A continuation carrier lets a fold's step be synthesized as its own problem. An inductive equation family lets an arithmetic proof use mutually supporting facts rather than demand individual conservation. A rank on a tagged obligation graph lets recursive proof structure be turned into ordinary well-founded induction.

The proposed design deliberately uses two algebraic lanes. The linear lane is predictable, degree-bounded, and complete in its affine fragment. The ideal lane is target-directed and handles polynomial multipliers and degree growth, at the cost of potentially expensive algebra. Cyclic compilation then consumes their proved local facts without accepting a circular argument.

The immediate implementation priority is to lower the already executable exact certificates to Lean, not to enlarge the untrusted search first. Once that gate works, the continuation and cyclic extensions have sharply defined entry points, concrete regression targets, and evidence that their central algorithms behave as specified.

A Certificate conventions

A.1 Exact polynomials

The transport representation of a rational is the pair `[numerator, denominator]`, with positive denominator and relatively prime entries. A polynomial is a list of pairs `[exponentVector, rational]`. The zero polynomial is the empty list. Exponents are nonnegative integers; zero coefficients and repeated monomials are rejected. The variable arity is supplied by the externally owned problem, not inferred from a certificate.

For instance, in variable order (x, y) , $\frac{1}{2}x^2 - 3y$ is

```
[
  [[2, 0], [1, 2]],
  [[0, 1], [-3, 1]]
]
```

The checker need not require a particular monomial ordering for acceptance. The searcher uses a deterministic order for row reduction and graded lexicographic order for its Gröbner routine. These are algorithmic choices, not mathematical assumptions about the polynomial identities.

A.2 Action records

For `pullback-space-v1`, `basis` is a list of r polynomials, `actions[a][i][j]` is a rational scalar, and `target_coefficients[i]` is a rational scalar. For `pullback-ideal-v1`, `generators` replaces `basis`, and the entries of `actions` and `target_coefficients` are polynomials. The former is the constant-multiplier special case of the latter, but keeping separate tags makes malformed scalar/polynomial payloads easier to reject.

The problem owns initialization, transitions, and the target. The certificate cannot change the target by returning a different convenient equation. Empty or redundant invariant families may be valid, but they prove a nonzero target only when the checked target combination actually reconstructs it.

A.3 Ranking records

An affine expression is encoded constant-first:

$$[b, w_1, \dots, w_n] \longleftrightarrow b + \sum_i w_i x_i.$$

Every `layers[k]` maps every node tag to its affine rank at that layer. Every edge has one witness record with its pivot, a decrease combination, and one nonnegativity combination per updated coordinate. Multipliers are ordered against explicit edge guards, then $x_1, \dots, x_n$, then 1.

The two-phase example uses

```
"layers": [ { "P": [1, 2], "Q": [0, 2] } ]
```

The checker verifies all edges in their original order. Dropping a hard edge from the certificate is rejected by the coverage check; adding another easy edge's witness in its place fails the exact coefficient identity.

A.4 Orbit records

An orbit counterexample contains rational initialization parameters and a list of transition indices in *execution order*. It is checked by direct exact evaluation of the problem's maps. For the noncommuting fixture,

$$F_0(x, y) = (y, 0), \quad F_1(x, y) = (0, 1), \quad (x, y)_0 = (0, 0),$$

the target is $x = 0$. Word $[1, 0]$ reaches $(1, 0)$ and violates it; word $[0, 1]$ ends at $(0, 1)$ and does not. This is why reversing the word is a meaningful corruption rather than another valid counterexample.

B Why the finite synthesis counts have the reported shape

With n distractor element binders, the four hole grammars have sizes $n + 1, n + 2, n + 2, n + 8$. The first locally valid base is last, the valid negative branch is last, the valid zero branch is second-to-last, and the valid positive continuation application is last. In the fixed tuple order, the successful complete term therefore has ordinal

$$(n + 1)(n + 2)^2(n + 8) - (n + 8).$$

For $n = 12$ this is 50 940. Local filtering tests every component once, so its component count is

$$(n + 1) + 2(n + 2) + (n + 8) = 4n + 13,$$

and each hole retains one candidate, yielding one assembled term.

The contrast is quartic versus linear in this deliberately controlled family. It is not a complexity theorem for unrestricted synthesis. Its assumptions are visible: fixed sketch, independent exact local contracts, finite grammars, distinct distractors, and the stated order. Correlated holes need a join, and local contract discovery itself can be expensive.

C Reproducing the package

From the archive root, use:

```
python -S -m unittest discover -s prototype -p "test_*.py" -v
python -S prototype/run_experiments.py --out reproduced-results
python -S prototype/replay.py reproduced-results/certificates.json
```

The optional independent CAS comparison is:

```
python prototype/validate_sympy.py --out reproduced-results/sympy-validation.json
```

It requires SymPy; the recorded comparison used version 1.14.0. The other commands do not. The archived results/ directory contains the original evidence, while reruns default to reproduced-results/ to avoid overwriting it.

To rebuild this article:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error main.tex
pdflatex -interaction=nonstopmode -halt-on-error main.tex
pdflatex -interaction=nonstopmode -halt-on-error main.tex
```

The distributed PDF is named `forge-extensions.pdf`. A fresh build produces `main.pdf`; the build script copies it to the distributed name. The required LaTeX packages are listed by the preamble. No font files are distributed.

C.1 A first Lean verification session

In a project with the repository's pinned Lean toolchain, the first experiment should be to compile `lean/Specimens.lean`, inspect the printed axiom sets, and retain the compiler output as a new receipt. This archive supplies no such receipt. Next, lower one small invariant certificate to a theorem using the pinned Mathlib environment and prove the source reification lemma for that exact fixture. Only then count it as a Lean acceptance.

The proof specimens cover fold correctness for an explicit list encoding, invariant propagation along a reachability derivation, and closing recursive calls by a well-founded induction hypothesis. They deliberately do not define a pretend `forge` tactic that succeeds by calling an unimplemented backend.

References

- [1] Vladimir Reshetnikov. *Forge repository*, revision 674521027d968d59f7b83220ed52304a85cb55e2. README and source inventory, inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/tree/674521027d968d59f7b83220ed52304a85cb55e2>.
- [2] Forge. *Structural search*, merged article, `article/sections/05-structural.tex`, at the Forge revision above. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/05-structural.tex>.
- [3] Forge. *Provenance and evidence*, merged article, `article/sections/B-provenance.tex`, at the Forge revision above. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/B-provenance.tex>.
- [4] Forge. *Ideas from Leant and Djex*, `docs/IDEAS-FROM-LEANT-DJEX.md`, at the Forge revision above. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [5] Forge. *Runtime design contract*, `lean/Forge/Design/Runtime.lean`, at the Forge revision above. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/lean/Forge/Design/Runtime.lean>.
- [6] Forge. *Lean integration*, merged article, `article/sections/08-integration.tex`, at the Forge revision above. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/08-integration.tex>.
- [7] Vladimir Reshetnikov. *Leant repository*, revision 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/tree/6bf05ad78c467989e68290f2d08bbbed40802d485>.
- [8] Leant. *Indexing composition diagnostics*, 14 September 2026, `docs/reports/2026-09-14-indexing-composition-diagnostics.md`, at the Leant revision above. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/reports/2026-09-14-indexing-composition-diagnostics.md>.
- [9] Vladimir Reshetnikov. *Rank-N and impredicative types*, Djex, revision e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5, `docs/rank-n.md`. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/docs/rank-n.md>.
- [10] Peter-Michael Osera and Steve Zdancewic. *Type-and-Example-Directed Program Synthesis*. PLDI 2015. <https://www.cis.upenn.edu/~stevez/papers/OZ15.pdf>. DOI: 10.1145/2737924.2738007.
- [11] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. PLDI 2016. <https://arxiv.org/abs/1510.08419>.
- [12] Michael Karr. *Affine Relationships Among Variables of a Program*. Acta Informatica 6, 133–151, 1976. DOI: 10.1007/BF00268497.
- [13] Enric Rodríguez-Carbonell and Deepak Kapur. *An Abstract Interpretation Approach for Automatic Generation of Polynomial Invariants*. SAS 2004. <https://www.cs.unm.edu/~kapur/mypapers/sas04.pdf>. DOI: 10.1007/978-3-540-27864-1_21.
- [14] Erdenebayer Bayarmagnai, Fatemeh Mohammadi, and Rémi Prébet. *Algebraic and Algorithmic Methods for Computing Polynomial Loop Invariants*. Extended article, arXiv:2412.14043, version 2, inspected 15 September 2026. <https://arxiv.org/html/2412.14043v2>. The earlier ISSAC 2024 article is titled *Algebraic Tools for Computing Polynomial Loop Invariants*.
- [15] Julian Müllner, Marcel Moosbrugger, and Laura Kovács. *Strong Invariants Are Hard: On the Hardness of Strongest Polynomial Invariants for (Probabilistic) Programs*. 2023. <https://arxiv.org/abs/2307.10902>.

- [16] Eddie Jones, C.-H. Luke Ong, and Steven Ramsay. *CycleQ: An Efficient Basis for Cyclic Equational Reasoning*. arXiv:2111.12553, 2021. <https://arxiv.org/abs/2111.12553>.
- [17] Lide Grotenhuis and Daniël Otten. *Unravelling Abstract Cyclic Proofs into Proofs by Induction*. arXiv:2602.12054, 2026. <https://arxiv.org/abs/2602.12054>.
- [18] The Mathlib community. *Multivariate polynomial evaluation*, Mathlib/Algebra/MvPolynomial/Eval.lean. Revision 1cf325a0cf67aca2b04d76b5380ff6a9e410ae4a. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410ae4a/Mathlib/Algebra/MvPolynomial/Eval.lean>.
- [19] Python-FLINT project. *fmpq_mat: matrices over rational numbers*. Documentation inspected 15 September 2026. https://python-flint.readthedocs.io/en/latest/fmpq_mat.html.
- [20] Python-FLINT project. *fmpz_mat: matrices over integers*. Documentation inspected 15 September 2026. https://python-flint.readthedocs.io/en/latest/fmpz_mat.html.
- [21] Singular project. *Ideal-related functions: lift and liftstd*. Official manual, documented interface. https://www.singular.uni-kl.de/Manual/4-0-3/sing_89.htm.
- [22] SymPy project. *Polynomial manipulation reference: Gröbner bases and reduction*. <https://docs.sympy.org/latest/modules/polys/reference.html>. The optional differential experiment used SymPy 1.14.0.
- [23] cvc5 project. *SMT and syntax-guided synthesis capabilities*. Official project documentation, inspected 15 September 2026. <https://cvc5.github.io/>.
- [24] Lean Language Reference. *Recursive definitions*, version 4.34.0 as displayed when inspected. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [25] Lean Language Reference. *Inductive types*. <https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/>.